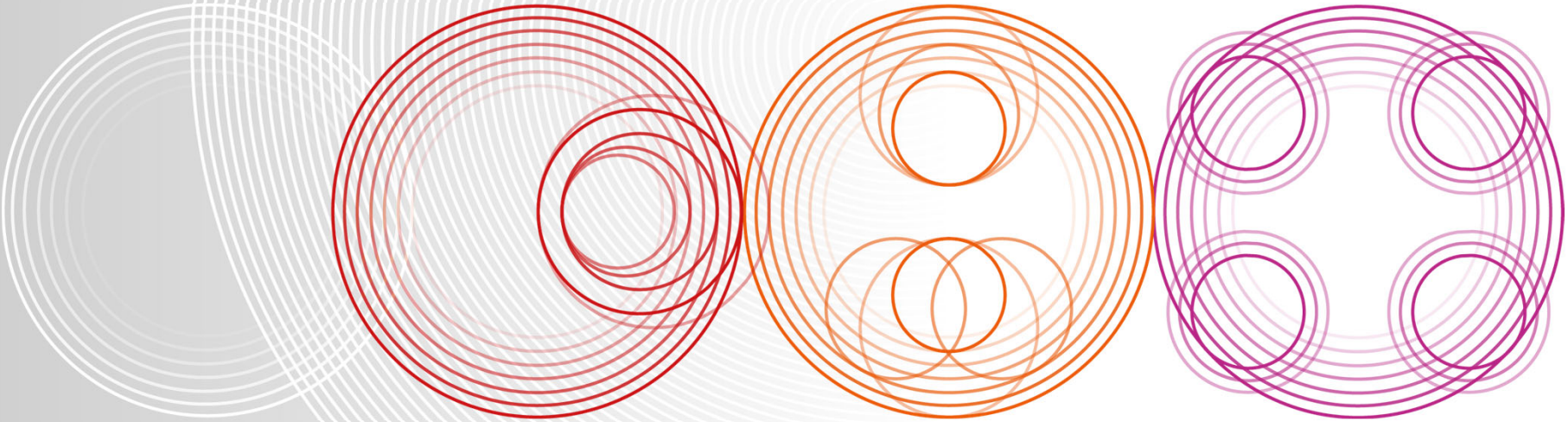
2026-10-15

Coding und Digitalisierung 003

Funktionen, Fehlerbehandlung, Pandas-Zeitreihen & OOP



Roter Faden der Vorlesung
Vom unstrukturierten Einzelskript zur robusten, industriellen Datenanalyse

Stunde 1 & Stunde 2: Grundlagen & Datenimport

- **Stunde 1 – Funktionen & Fehlerbehandlung (Clean Code):** Warum linearer Code mit Schleifen ab einer gewissen Größe scheitert.
  - Das DRY-Prinzip (Don't Repeat Yourself) & eigene Funktionen (def, return).
  - Gültigkeitsbereich (Scope) lokaler Variablen verstehen.
  - Abstürze durch Sensorfehler ('Error') mit try / except ValueError abfangen.
- **Stunde 2 – Pandas DataFrames & Datenimport:** Warum Tabellen mehr brauchen als reine NumPy-Matrizen.
  - Der DataFrame als programmierbare Excel-Tabelle.
  - Große Datensätze (bb12024.csv) mit pd.read_csv() laden und mit head(), info() sowie describe() untersuchen.

Stunde 3 & Stunde 4: Zeitreihen & Architektur (OOP)

- **Stunde 3 – Datenbereinigung & Resampling (Zeitreihen):** Reale Sensordaten für Analysen vorbereiten.
  - **Umgang mit Datenlücken:** df.dropna(), df.fillna() und Interpolation.
  - Zeitstempel konvertieren (pd.to_datetime) und als Index setzen.
  - **Asynchrone Sensoren exakt auf ein 5-Minuten-Raster takten:** df.resample('5min').mean().
- **Stunde 4 – Objektorientierte Programmierung (OOP) & Synthese:** Komplexe Systeme strukturiert abbilden.
  - Klassen (Bauplan) und Objekte (Instanzen) mit __init__ und self.
  - Kombination aus OOP, Fehlerbehandlung und Pandas für das Semesterprojekt.

📌 **Fazit: Jede Vorlesungsstunde schließt mit einer praktischen Live-Coding-Zwischenübung ab.**

STUNDE 1 (45 Min.)

Funktionen und Fehlerbehandlung (Clean Code)

Wiederverwendbarer Code nach dem DRY-Prinzip und Schutz vor Programmabstürzen

Lernziele & Schwerpunkte dieser Vorlesungsstunde:

- **Motivation:** Warum linearer Code mit kopierten Schleifen bei wachsenden Projekten unwartbar wird.
- **Das DRY-Prinzip (Don't Repeat Yourself):** Logik kapseln und zentral pflegen.
- **Funktionen definieren:** Aufbau mit def, Parametern, Einrückung, Docstring und return.
- **Gültigkeitsbereich (Scope):** Warum lokale Variablen wie 'summe' außerhalb der Funktion unsichtbar sind.
- **Fehlerbehandlung:** Sensor-Störungen ('Error' statt Float) sicher mit try und except ValueError abfangen.

Motivation: Das Problem mit linearem Copy-Paste-Code

Warum Schleifen allein ab einer gewissen Projektgröße nicht mehr ausreichen

Typische Probleme in gewachsenen Skripten

- **Redundanz (WET – We Enjoy Typing):** Für jeden neuen Sensor wird derselbe Schleifenblock kopiert und leicht angepasst.
- **Hohe Fehleranfälligkeit:** Beim Kopieren wird schnell ein Variablenname übersehen (z. B. teilt man durch die falsche Listenlänge).
- **Schlechte Wartbarkeit:** Ändert sich die Berechnungslogik (z. B. Rundung auf 2 Nachkommastellen), muss man 20 Stellen im Skript suchen.
- **Unübersichtlichkeit:** Ein Hauptprogramm mit 500 Zeilen Schleifen-Code ist kaum noch lesbar oder im Team testbar.

Das DRY-Prinzip (Don't Repeat Yourself) – Jede Logik sollte im gesamten Programm nur an einer einzigen Stelle existieren!

negativbeispiel_spaghetti.py

```
# Messreihen aus drei Becken (O2 in mg/L)
becken_1 = [4.2, 4.5, 4.1, 4.4]
becken_2 = [1.8, 2.1, 1.9, 2.0]
becken_3 = [0.5, 0.4, 0.6, 0.5]

# Mittelwert Becken 1 (manuelle Schleife)
summe_1 = 0.0
for wert in becken_1:
    summe_1 += wert
mw_1 = summe_1 / len(becken_1)

# Mittelwert Becken 2 (Copy & Paste!)
summe_2 = 0.0
for wert in becken_2:
    summe_2 += wert
mw_2 = summe_2 / len(becken_1)  # BUG: becken_1 statt 2!

# ... bei 50 Sensoren wird das Skript unwartbar!
```


Arts Sciences
TH Köln

Anatomie einer Funktion in Python

Grundaufbau: def, Funktionsname, Parameter, Einrückung und return

Die 5 Bausteine einer Funktionsdefinition

- **Schlüsselwort def:** Signalisiert Python, dass hier ein neuer wiederverwendbarer Code-Baustein definiert wird.
- **Funktionsname:** Sprechender Name im snake_case (z. B. berechne_mittelwert), der beschreibt, was die Funktion tut.
- **Parameterliste in Klammern:** Die Eingabedaten (Input), mit denen die Funktion arbeiten soll, gefolgt von einem Doppelpunkt (:).
- **Eingerückter Funktionskörper:** Alle Zeilen mit 4 Leerzeichen Einrückung gehören zur Funktion.
- **Schlüsselwort return:** Beendet die Funktion und gibt das berechnete Ergebnis an den Aufrufer zurück.

Wichtig: Die Definition mit 'def' führt den Code noch nicht aus! Sie legt lediglich den Bauplan im Speicher ab.

anatomie_funktion.py

```
def berechne_mittelwert(liste):
    """Berechnet das arithmetische Mittel einer
Zahlenliste."""
    # 1. Sonderfall leere Liste abfangen
    if len(liste) == 0:
        return 0.0

    # 2. Berechnung über Schleife
    summe = 0.0
    for wert in liste:
        summe += wert

    mittelwert = summe / len(liste)

    # 3. Ergebnis zurückgeben
    return mittelwert
```


Arts Sciences
TH Köln

Live-Coding: berechne_mittelwert(liste) im Einsatz

Einmal schreiben, beliebig oft für unterschiedliche Sensordaten aufrufen

Vom Funktionsaufruf zum Rückgabewert

- **Trennung von Definition und Aufruf:** Oben im Skript stehen die Funktionen, darunter das eigentliche Hauptprogramm.
- **Argumentübergabe:** Beim Aufruf berechne_mittelwert(becken_1) wird die Liste becken_1 an den Parameter liste übergeben.
- **Wiederverwendbarkeit:** Egal ob 3 oder 300 Sensoren – jede Berechnung benötigt im Hauptprogramm nur noch eine einzige Zeile.
- **Zentrale Anpassung:** Möchten wir das Ergebnis auf 2 Nachkommastellen runden, ändern wir nur eine Zeile in der Funktion.

💡 **Praxis-Tipp / Merksatz:**
Ohne 'return' gibt eine Python-Funktion automatisch den Wert 'None' zurück – print() in der Funktion ersetzt kein return!

live_coding_mittelwert.py

```
def berechne_mittelwert(liste):
    summe = 0.0
    for wert in liste:
        summe += wert
    return round(summe / len(liste), 2)

# --- HAUPTPROGRAMM ---
becken_1 = [4.2, 4.5, 4.1, 4.4]
becken_2 = [1.8, 2.1, 1.9, 2.0]
zulauf_temp = [14.2, 14.5, 14.8, 14.1, 14.4]

mw_b1 = berechne_mittelwert(becken_1)
mw_b2 = berechne_mittelwert(becken_2)
mw_temp = berechne_mittelwert(zulauf_temp)

print(f'O2 Becken 1: {mw_b1} mg/L')
print(f'O2 Becken 2: {mw_b2} mg/L')
print(f'Zulauf-Temp: {mw_temp} °C')

[Ausgabe]
>>> O2 Becken 1: 4.3 mg/L
>>> O2 Becken 2: 1.95 mg/L
>>> Zulauf-Temp: 14.4 °C
```


Arts Sciences
TH Köln

Default-Parameter & mehrere Rückgabewerte

Funktionen flexibel gestalten am Beispiel der Ausreißer-Filterung

Standardwerte und Tupel-Rückgaben

- **Default-Parameter:** Mit schwellenwert=50.0 definieren wir einen Standardwert direkt im Funktionskopf.
- **Optionaler Aufruf:** Wird kein zweites Argument übergeben, nutzt Python automatisch die 50.0; andernfalls wird sie überschrieben.
- **Keyword-Argumente:** Aufrufe wie filtere_ausreisser(daten, schwellenwert=10.0) erhöhen die Lesbarkeit enorm.
- **Mehrere Werte zurückgeben:** Python kann mit return mw, min_w, max_w mehrere Kennzahlen gleichzeitig als Tupel liefern.

💡 **Praxis-Tipp / Merksatz:**
Regel: Parameter ohne Standardwert müssen im Funktionskopf immer VOR Parametern mit Default-Wert stehen!

default_parameter_praxis.py

```
def filtere_ausreisser(daten, schwellenwert=50.0):
    """Behält nur Messwerte <= schwellenwert."""
    bereinigt = []
    for w in daten:
        if w <= schwellenwert:
            bereinigt.append(w)
    return bereinigt

rohdaten = [4.1, 4.3, 999.9, 4.0, 85.2, 4.2]

# 1. Aufruf MIT Standardwert (50.0)
ok_standard = filtere_ausreisser(rohdaten)
print('Standard (<=50):', ok_standard)

# 2. Aufruf mit EIGENEM Schwellenwert (10.0)
ok_streng = filtere_ausreisser(rohdaten, schwellenwert=10.0)
print('Streng (<=10):  ', ok_streng)

[Ausgabe]
>>> Standard (<=50): [4.1, 4.3, 4.0, 4.2]
>>> Streng (<=10):   [4.1, 4.3, 4.0, 4.2]
```


Arts Sciences
TH Köln

Gültigkeitsbereich (Scope): Lokal vs. Global

Warum die Variable 'summe' außerhalb der Funktion nicht abrufbar ist

Das Black-Box-Prinzip von Funktionen

- **Lokaler Scope (Funktions-Namensraum):** Variablen wie summe oder wert, die innerhalb einer Funktion angelegt werden, sind lokal.
- **Lebensdauer:** Lokale Variablen entstehen beim Aufruf der Funktion und werden nach dem return sofort aus dem Speicher gelöscht.
- **Warum ist das gut?:** Verschiedene Funktionen können intern problemlos denselben Variablennamen summe nutzen, ohne sich gegenseitig zu überschreiben.
- **NameError verstehen:** Greift das Hauptprogramm direkt auf summe zu, wirft Python einen NameError: name 'summe' is not defined.

💡 **Praxis-Tipp / Merksatz:**
Best Practice: Funktionen sollten niemals globale Variablen heimlich verändern, sondern alles über Parameter & return austauschen.

scope_experiment.py

```
anlagen_name = 'Klärwerk Nord'  # GLOBALE Variable

def berechne_mittelwert(liste):
    summe = 0.0                 # LOKALE Variable
    for wert in liste:
        summe += wert
    return summe / len(liste)

messwerte = [2.0, 4.0, 6.0]
ergebnis = berechne_mittelwert(messwerte)

print('Mittelwert:', ergebnis)  # Funktioniert (4.0)
print('Summe:', summe)          # FEHLER!

[Ausgabe]
>>> Mittelwert: 4.0
>>> Traceback (most recent call last):
>>>   File 'scope_experiment.py', line 13
>>> NameError: name 'summe' is not defined
```


Arts Sciences
TH Köln

Wenn Sensoren 'Error' senden: Programmabsturz!

Warum reale Messdaten ohne Fehlerbehandlung jedes Skript zum Stehen bringen

Reale Felddaten sind niemals perfekt

- **Typische Störungen im Feld:** Kabelbruch, EMV-Störung, Sensor-Kalibrierung oder Timeout auf dem Feldbus.
- **Strings statt Zahlen:** Statt '4.25' liefert die Schnittstelle plötzlich Texte wie 'Error', 'NaN', '---' oder 'SENSOR_FAULT'.
- **Harter Absturz bei Typumwandlung:** Die Funktion float('Error') kann den Text nicht in eine Kommazahl übersetzen.
- **Folge im Dauerbetrieb:** Python bricht mit einem unbehandelten ValueError sofort ab – alle nachfolgenden Messwerte gehen verloren!

In industriellen Steuerungen und Datenloggern darf ein einzelner defekter Messwert niemals das Gesamtsystem stoppen. Das gleiche gilt für wissenschaftliche oder wirtschaftliche Analysen.

crash_ohne_schutz.py

```
# Rohdaten-Stream von einer seriellen Schnittstelle
sensor_stream = ['4.2', '4.5', 'Error', '4.1', '4.3']

gueltige_werte = []

for roh_text in sensor_stream:
    # Kritische Stelle: Umwandlung String -> Float
    messwert = float(roh_text)
    gueltige_werte.append(messwert)

print('Fertig eingelesen:', gueltige_werte)

[Ausgabe]
>>> Traceback (most recent call last):
>>>   File 'crash_ohne_schutz.py', line 8
>>>     messwert = float(roh_text)
>>> ValueError: could not convert string to float: 'Error'
# Programm bricht beim 3. Wert ab – Wert 4 und 5 fehlen!
```


Arts Sciences
TH Köln

Fehlerbehandlung mit try-except
Ausnahmen gezielt abfangen und mit sauberem Funktionsdesign kombinieren

- **Das try-except-Prinzip:** Im try-Block steht der kritische Code (z. B. float(x)). Tritt ein ValueError auf, springt Python sofort in den except-Block.
- **Gezieltes Abfangen:** Immer die konkrete Fehlerart (except ValueError:) angeben, um echte Programmierfehler nicht zu verschlucken.
- **Zwischenübung 1 (10 Min.):** Schreiben Sie eine Funktion berechne_sicheren_mittelwert(roh_liste), die eine Liste aus Strings entgegennimmt.
  - Wandeln Sie jeden Eintrag per try / except ValueError in float um.
  - Ignorieren Sie fehlerhafte Einträge ('Error', 'TIMEOUT') mit einer kurzen Warnmeldung und geben Sie den Mittelwert zurück.

Durch die Kapselung in einer Funktion mit try/except läuft das Hauptprogramm trotz 2 Sensorfehlern stabil weiter!

loesung_stunde1_try_except.py

```
def berechne_sicheren_mittelwert(roh_liste):
    saubere_werte = []
    for eintrag in roh_liste:
        try:
            wert = float(eintrag)
            saubere_werte.append(wert)
        except ValueError:
            print(f'Warnung: Ignoriere "{eintrag}"')

    if len(saubere_werte) == 0:
        return 0.0
    return round(sum(saubere_werte) / len(saubere_werte), 2)

# Test mit fehlerhaftem Sensor-Stream
stream = ['4.2', '4.5', 'Error', '4.1', 'TIMEOUT', '4.4']
mw = berechne_sicheren_mittelwert(stream)
print(f'Sicherer Mittelwert: {mw} mg/L')

[Ausgabe]
>>> Warnung: Ignoriere "Error"
>>> Warnung: Ignoriere "TIMEOUT"
>>> Sicherer Mittelwert: 4.3 mg/L
```


Arts Sciences
TH Köln

STUNDE 2 (45 Min.)

Pandas: DataFrames und Datenimport

Von einfachen Listen zur professionellen Tabellen- und Prozessdatenanalyse

Lernziele & Schwerpunkte dieser Vorlesungsstunde:

- **Von Listen & NumPy zu Tabellen:** Warum heterogene Messdaten (Datum, Text, Float) nach Pandas verlangen.
- **Der DataFrame:** Die programmierbare Excel-Tabelle für Python mit Zeilenindex und Spaltennamen.
- **Datenimport in der Praxis:** Große CSV-Dateien wie 'bb12024.csv' mit pd.read_csv() einlesen.
- **Erste Inspektion:** Struktur, Datentypen und Fehlwerte mit df.head(), df.info() und df.describe() prüfen.
- **Gezielter Zugriff:** Spalten selektieren und kritische Prozesszustände über Bedingungen filtern.

Von Listen und NumPy zu echten Datentabellen

Warum NumPy perfekt für Matrizen ist, aber bei realen Messprotokollen an Grenzen stößt

Python-Listen & NumPy-Arrays

- **Python-Listen:** Flexibel, aber bei 100.000 Messzeilen extrem langsam und ohne eingebaute Tabellenoperationen.
- **NumPy-Arrays (ndarray):** Extrem schnell für reine Vektor- und Matrizenrechnung (Lineare Algebra, Signalverarbeitung).
- **Homogener Datentyp-Zwang in NumPy:** Alle Elemente einer Matrix müssen denselben Datentyp haben (z. B. nur float64).
- **Fehlende Beschriftung:** Zugriff erfolgt nur über nackte Positionsindizes wie daten[0, 3] statt über sprechende Spaltennamen.
- **Problem bei Messdaten:** Eine CSV-Datei enthält Zeitstempel ('2024-02-01 08:00'), Sensor-IDs ('BB1') und Messwerte (4.21) gleichzeitig.

Pandas (Panel Data) als Branchenstandard

- **Baut direkt auf NumPy auf:** Nutzt unter der Haube die hohe C-Geschwindigkeit von NumPy-Arrays.
- **Heterogene Datentypen:** Jede Spalte darf einen eigenen Datentyp besitzen (datetime64, float64, int64, string/object).
- **Sprechende Spaltennamen & Indizes:** Zugriff über df['o2_gehalt'] oder über echte Zeitstempel statt unverständlicher Indexnummern.
- **Integrierte Datenbereinigung:** Direkter Umgang mit fehlenden Werten (NaN), Duplikaten und fehlerhaften Zeitachsen.
- **Mächtiger I/O-Werkzeugkasten:** Liest und schreibt CSV, Excel, Parquet, SQL-Datenbanken und JSON mit einer einzigen Codezeile.

Faustregel: Reine Mathematik & Matrizen -> NumPy | Tabellarische Messdaten & Zeitreihen -> Pandas!

Der Pandas DataFrame: Die Excel-Tabelle für Python

Aufbau aus Index (Zeilen), Columns (Spalten) und Series (Einzelspalten)

Anatomie eines DataFrames

- **Import-Konvention:** Pandas wird weltweit einheitlich mit import pandas as pd eingebunden.
- **DataFrame (2D-Tabelle):** Besteht aus benannten Spalten (Columns) und einem Zeilenindex (Index, standardmäßig 0, 1, 2, ...).
- **Series (1D-Spalte):** Jede einzelne Spalte eines DataFrames ist technisch gesehen ein Pandas-Series-Objekt.
- **Vorteil gegenüber Excel:** Verarbeitet problemlos Millionen Zeilen, und jeder Bearbeitungsschritt ist im Skript 100 % reproduzierbar.

💡 **Praxis-Tipp / Merksatz:**
Links stehen die Zeilenindizes (0, 1, 2), oben die Spaltennamen ('zeit', 'becken', 'o2_mg_l', 'temp_c').

erster_dataframe.py

```
import pandas as pd

# DataFrame aus einem Python-Dictionary erzeugen
messdaten = {
    'zeit': ['08:00', '08:05', '08:10'],
    'becken': ['BB1', 'BB1', 'BB1'],
    'o2_mg_l': [4.12, 4.35, 3.98],
    'temp_c': [15.1, 15.2, 15.2]
}

df = pd.DataFrame(messdaten)
print(df)

[Ausgabe]
>>>     zeit becken  o2_mg_l  temp_c
>>> 0  08:00    BB1     4.12    15.1
>>> 1  08:05    BB1     4.35    15.2
>>> 2  08:10    BB1     3.98    15.2
```


Arts Sciences
TH Köln

Der Datenimport: pd.read_csv() in der Praxis

Große Prozessdatensätze wie 'bb12024.csv' zuverlässig einlesen

Worauf man beim CSV-Import achten muss

- **Standard-Aufruf:** Mit df = pd.read_csv('bb12024.csv') lädt Pandas eine Komma-getrennte Datei direkt in einen DataFrame.
- **Deutsche vs. Englische CSV-Dateien:** Deutsche Exporte (z. B. aus SPS/SCADA oder Excel) nutzen oft Semikolon (;) als Trennzeichen und Komma (,) als Dezimaltrennzeichen.
- **Wichtige Parameter:** Mit sep=';' und decimal=',' verhindert man, dass Kommazahlen fälschlicherweise als Text erkannt werden.
- **Attribut df.shape:** Verrät sofort die Dimension der Tabelle als Tupel (Anzahl_Zeilen, Anzahl_Spalten).

💡 **Praxis-Tipp / Merksatz:**
Ein komplettes Jahr im 10-Minuten-Takt hat bereits über 52.000 Zeilen – pd.read_csv() lädt das in wenigen Millisekunden!

datenimport_bb12024.py

```
import pandas as pd

# 1. Standard-Import (Komma, Punkt als Dezimalzeichen)
df = pd.read_csv('bb12024.csv')

# 2. Typischer Import für deutsche Industrie-Exporte:
# df = pd.read_csv('bb12024_de.csv', sep=';', decimal=',')

zeilen, spalten = df.shape
print('Datensatz erfolgreich geladen!')
print(f'Anzahl Messzeilen: {zeilen}')
print(f'Anzahl Spalten:    {spalten}')
print('Spaltennamen:', list(df.columns))

[Ausgabe]
>>> Datensatz erfolgreich geladen!
>>> Anzahl Messzeilen: 52560
>>> Anzahl Spalten:    5
>>> Spaltennamen: ['zeit', 'o2_ist', 'o2_soll',
>>>                'nh4_mg_l', 'temp_c']
```


Arts Sciences
TH Köln

Erster Blick in die Daten: df.head() und df.tail()

Stichprobenartige Sichtprüfung der ersten und letzten Zeilen von 'bb12024.csv'

Warum immer zuerst head() aufrufen?

- **Schnelle Plausibilitätsprüfung:** Wurden die Spaltenüberschriften korrekt erkannt oder stehen sie in der ersten Datenzeile?
- **df.head(n):** Zeigt die ersten n Zeilen des DataFrames an (ohne Argument standardmäßig die ersten 5 Zeilen).
- **df.tail(n):** Zeigt die letzten n Zeilen an – ideal, um zu prüfen, bis zu welchem Datum die Aufzeichnung reicht.
- **Früherkennung von Fehlern:** Man sieht sofort, wie Zeitstempel formatiert sind und ob bereits in den ersten Zeilen NaN-Werte auftauchen.

💡 **Praxis-Tipp / Merksatz:**
Schon ein einziger Blick auf df.head() zeigt uns zwei Aufgaben für Stunde 3: Fehlwerte (NaN) und unregelmäßige Zeitstempel!

sichtpruefung_head.py

```
import pandas as pd

df = pd.read_csv('bb12024.csv')

# Die ersten 5 Zeilen ausgeben
print(df.head())

[Ausgabe]
>>>                  zeit  o2_ist  o2_soll  nh4_mg_l  temp_c
>>> 0  2024-01-01 00:01:00    2.14      2.0      1.85    11.4
>>> 1  2024-01-01 00:04:00    2.08      2.0      1.88    11.4
>>> 2  2024-01-01 00:07:00     NaN      2.0      1.92    11.3
>>> 3  2024-01-01 00:11:00    1.95      2.0      1.95    11.3
>>> 4  2024-01-01 00:14:00    2.02      2.0      1.90    11.3

# Beobachtungen:
# 1. Zeile 2 hat einen fehlenden Wert (NaN) bei o2_ist!
# 2. Zeitabstände sind unregelmäßig (00:01, 00:04, 00:07)!
```


Arts Sciences
TH Köln

Röntgenblick in die Struktur: df.info()

Datentypen (Dtypes), Speicherbedarf und fehlende Werte (Non-Null Count) prüfen

Was uns df.info() auf einen Blick verrät

- **RangeIndex:** Zeigt die Gesamtanzahl der Einträge (z. B. 52560 entries, 0 to 52559).
- **Non-Null Count:** Vergleicht man diese Zahl mit der Gesamtzeilenzahl, erkennt man sofort, in welchen Spalten Messwerte fehlen!
- **Dtype (Datentyp der Spalte):** float64 steht für Kommazahlen, int64 für Ganzzahlen.
- **Warnzeichen 'object':** Steht bei einer Zahlen- oder Datumsspalte der Typ object, liegen die Daten noch als unformatierter Text (String) vor!

💡 **Praxis-Tipp / Merksatz:**
Diagnose: 'zeit' ist noch vom Typ 'object' (Text), und bei 'o2_ist' fehlen genau 450 Messwerte (52560 - 52110)!

struktur_info.py

```
import pandas as pd

df = pd.read_csv('bb12024.csv')
df.info()

[Ausgabe]
>>> <class 'pandas.core.frame.DataFrame'>
>>> RangeIndex: 52560 entries, 0 to 52559
>>> Data columns (total 5 columns):
>>>  #   Column    Non-Null Count  Dtype
>>> ---  ------    --------------  -----
>>>  0   zeit      52560 non-null  object
>>>  1   o2_ist    52110 non-null  float64
>>>  2   o2_soll   52560 non-null  float64
>>>  3   nh4_mg_l  51980 non-null  float64
>>>  4   temp_c    52560 non-null  float64
>>> dtypes: float64(4), object(1)
>>> memory usage: 2.0+ MB
```


Arts Sciences
TH Köln

Statistische Kurzanalyse mit df.describe()

Mittelwert, Standardabweichung, Minimum, Maximum und Quartile sofort berechnen

Automatische Kennzahlen für alle Zahlenspalten

- **Ein Befehl statt 10 Schleifen:** df.describe() berechnet für jede numerische Spalte die wichtigsten statistischen Kennwerte.
- **mean & std:** Arithmetischer Mittelwert und Standardabweichung (Streuung der Messwerte).
- **min & max:** Ideal, um physikalisch unmögliche Ausreißer (z. B. -99.9 bei Konzentrationen oder 999.0 bei Defekten) sofort zu entlarven.
- **25%, 50% (Median), 75%:** Die Perzentile zeigen die robuste Verteilung der Messdaten unabhängig von extremen Ausreißern.

statistik_describe.py

```
import pandas as pd

df = pd.read_csv('bb12024.csv')
print(df.describe().round(2))

[Ausgabe]
>>>          o2_ist   o2_soll  nh4_mg_l    temp_c
>>> count  52110.00  52560.00  51980.00  52560.00
>>> mean       1.84      2.00      2.45     15.62
>>> std        0.65      0.35      1.78      4.12
>>> min        0.02      0.00     -0.50      8.10
>>> 25%        1.45      2.00      1.10     12.20
>>> 50%        1.92      2.00      2.05     15.40
>>> 75%        2.15      2.20      3.40     19.10
>>> max        8.40      3.50     14.20     24.80
```

💡 **Praxis-Tipp / Merksatz:**
Auffälligkeit im Output: 'nh4_mg_l' hat ein Minimum von -0.50 mg/L! Negative Konzentrationen sind ein typischer Kalibrierfehler.

Arts Sciences
TH Köln

Spaltenzugriff, Filterung & Zwischenübung 2

Gezielte Abfragen auf dem DataFrame formulieren (Boolean Indexing)

Stunde 2 | Filterung & Übung

🛠️ Aufgabenstellung (Live-Übung):

- **Spalten auswählen:** Eine einzelne Spalte ruft man mit df['o2_ist'] ab, mehrere Spalten mit einer Liste df[['zeit', 'o2_ist']].
- **Zeilen filtern (Boolean Indexing):** Mit df[df['nh4_mg_l'] > 5.0] erhält man alle Zeilen, in denen die Ammonium-Last hoch ist.
- **Zwischenübung 2 (10 Min.):** Laden Sie die Datei bb12024.csv in einen DataFrame df.
  - Geben Sie die ersten 3 Zeilen und die Struktur (info) aus.
  - Berechnen Sie den Mittelwert der Spalte 'temp_c'.
  - Filtern Sie alle Zeilen heraus, in denen 'o2_ist' unter 0.5 mg/L gefallen ist, und geben Sie deren Anzahl aus.

⏱️ **Ziel & Lösungshinweis:**
Merke: df['spalte'].mean() ignoriert fehlende Werte (NaN) automatisch, ohne dass das Programm abstürzt!

loesung_stunde2_pandas.py

```
import pandas as pd

# 1. CSV-Datei laden und sichten
df = pd.read_csv('bb12024.csv')
print(df.head(3))

# 2. Einzelne Spalte auswerten
mittel_temp = df['temp_c'].mean()
print(f'Mittlere Temperatur: {mittel_temp:.2f} °C')

# 3. Kritische Sauerstoff-Zustände filtern (< 0.5 mg/L)
kritisch_df = df[df['o2_ist'] < 0.5]
print(f'Anzahl kritischer Messpunkte: {len(kritisch_df)}')

[Ausgabe]
>>>                  zeit  o2_ist  o2_soll  nh4_mg_l  temp_c
>>> 0  2024-01-01 00:01:00    2.14      2.0      1.85    11.4
>>> 1  2024-01-01 00:04:00    2.08      2.0      1.88    11.4
>>> 2  2024-01-01 00:07:00     NaN      2.0      1.92    11.3
>>> Mittlere Temperatur: 15.62 °C
>>> Anzahl kritischer Messpunkte: 1420
```


Arts Sciences
TH Köln

STUNDE 3 (45 Min.)

Pandas: Datenbereinigung & Zeitreihen-Resampling

Fehlwerte bereinigen, Zeitachsen indizieren und asynchrone Sensoren synchronisieren

Lernziele & Schwerpunkte dieser Vorlesungsstunde:

- **Umgang mit Fehlwerten (NaN):** Wann löschen wir Zeilen mit df.dropna(), wann schließen wir Lücken mit df.fillna()?
- **Plausibilitätsfilterung:** Unphysikalische Messwerte und eingefrorene Sensorzustände ausschließen.
- **Zeitreihen-Fundament:** Text-Zeitstempel mit pd.to_datetime() in echte Zeit-Objekte umwandeln.
- **DatetimeIndex:** Die Zeitachse mit df.set_index('zeit') zum Herzstück des DataFrames machen.
- **Resampling auf 5 Minuten:** Asynchrone Sensordaten (08:01, 08:04, 08:07) exakt über df.resample('5min').mean() takten.

Fehlende Werte (NaN) in Messdaten diagnostizieren

Wie entstehen Datenlücken und wie spürt man sie systematisch auf?

Stunde 3 | Fehlwerte

Ursachen und Diagnose von NaN (Not a Number)

- **Woher kommen Lücken?:** Sensor-Reinigung, kurzzeitiger Stromausfall, Funk-Abbruch oder herausgefilterte 'Error'-Strings beim CSV-Import.
- **Darstellung in Pandas:** Fehlende Werte werden einheitlich als NaN (Not a Number) markiert.
- **Diagnose mit df.isna().sum():** Zählt für jede Spalte exakt aus, wie viele NaN-Einträge vorhanden sind.
- **Warum bereinigen?:** Viele nachfolgende Algorithmen (z. B. Regler-Modelle, Korrelationen oder Machine Learning) akzeptieren keine NaN-Lücken.

💡 **Praxis-Tipp / Merksatz:**
Vor jeder Bereinigung gilt: Erst mit df.isna().sum() prüfen, wie groß das Problem überhaupt ist!

diagnose_fehlwerte.py

```
import pandas as pd

df = pd.read_csv('bb12024.csv')

# Anzahl fehlender Werte pro Spalte zählen
fehlwerte = df.isna().sum()
print('--- Fehlende Werte pro Spalte ---')
print(fehlwerte)

# Prozentualer Anteil der Datenlücken
prozent = (df.isna().sum() / len(df)) * 100
print('\n--- Anteil in Prozent (%) ---')
print(prozent.round(2))

[Ausgabe]
>>> --- Fehlende Werte pro Spalte ---
>>> zeit          0
>>> o2_ist      450
>>> o2_soll       0
>>> nh4_mg_l    580
>>> temp_c        0
```


Arts Sciences
TH Köln

Strategien gegen Datenlücken: dropna() vs. fillna()

Zeilen komplett verwerfen oder Lücken intelligent auffüllen?

Methode 1: Zeilen löschen mit df.dropna()

- **Funktionsweise:** df_clean = df.dropna() entfernt jede Zeile, in der mindestens ein einziger Wert fehlt (NaN).
- **Gezieltes Löschen:** Mit df.dropna(subset=['o2_ist']) werden nur Zeilen gelöscht, in denen konkret der O2-Wert fehlt.
- **Vorteil:** Es werden keinerlei künstliche oder geschätzte Messwerte erfunden.
- **Nachteil bei Zeitreihen:** Es entstehen zeitliche Löcher in der Tabelle – aus einem regelmäßigen Takt wird eine lückenhafte Zeitreihe.

Methode 2: Lücken schließen mit df.fillna() / interpolate()

- **Fester Ersatzwert:** df.fillna(0.0) ersetzt jedes NaN durch 0.0 (Vorsicht: Bei Temperaturdaten verfälscht 0 °C den Mittelwert massiv!).
- **Forward Fill (Halteglied):** df.ffill() schreibt den letzten gültigen Messwert fort, bis ein neuer Messwert eintrifft (wie in einer SPS).
- **Lineare Interpolation:** df.interpolate() zieht eine Gerade zwischen dem Wert vor und nach der Lücke – ideal für träge physikalische Größen.
- **Vorteil:** Die Zeitachse bleibt vollständig erhalten.

📌 **Fazit: Praxis-Regel: Kurze Lücken (1–3 Messpunkte) per ffill() oder interpolate() schließen; lange Ausfälle nicht blind auffüllen!**

Technology
Arts Sciences
TH Köln

Code-Beispiel: Fehlwerte & unplausible Daten bereinigen

Kombination aus Lückenfüllung und dem Ausschluss ungültiger Sensorzustände

Dreistufige Datenbereinigung im Code

- **Schritt 1 – Negative Fehlmessungen entfernen:** Konzentrationen wie nh4_mg_l können physikalisch nicht kleiner als 0 sein.
- **Schritt 2 – Kurze Lücken schließen:** Mit ffill() (oder interpolate()) werden einzelne NaN-Aussetzer überbrückt.
- **Schritt 3 – Verbleibende Ränder bereinigen:** Sollte direkt in Zeile 0 ein NaN gestanden haben, räumt ein abschließendes dropna() auf.
- **Kontrolle:** Ein erneuter Aufruf von df.isna().sum() bestätigt, dass der Datensatz nun zu 100 % lückenfrei ist.

💡 **Praxis-Tipp / Merksatz:**
Wichtig: Bei Zuweisungen wie df = df.dropna() speichern wir das bereinigte Ergebnis wieder in df zurück!

datenbereinigung_praxis.py

```
import pandas as pd

df = pd.read_csv('bb12024.csv')

# 1. Physikalisch unmögliche Werte (< 0) ausschließen
df = df[df['nh4_mg_l'] >= 0.0]

# 2. Kurze Sensor-Aussetzer mit letztem gültigen Wert füllen
df['o2_ist'] = df['o2_ist'].ffill()
df['nh4_mg_l'] = df['nh4_mg_l'].interpolate()

# 3. Falls am Dateianfang noch NaN übrig blieb -> löschen
df = df.dropna()

print('Verbleibende NaN-Werte:')
print(df.isna().sum())

[Ausgabe]
>>> Verbleibende NaN-Werte:
>>> zeit        0
>>> o2_ist      0
>>> o2_soll     0
>>> nh4_mg_l    0
>>> temp_c      0
```


Arts Sciences
TH Köln

Zeitreihen verstehen: Konvertierung mit pd.to_datetime()

Das wichtigste Werkzeug für Messdaten: Aus dummen Text-Strings echte Zeitstempel machen

Warum reicht ein String '2024-01-01 08:01' nicht?

- **Text kennt keine Zeitdauer:** Für einen normalen String ist '08:05' nur eine Buchstabenfolge – Python weiß nicht, dass 4 Minuten seit '08:01' vergangen sind.
- **Die Lösung pd.to_datetime():** Analysiert das Textformat und übersetzt jeden Eintrag in den hochpräzisen Datentyp datetime64[ns].
- **Rechnen mit Zeit:** Erst danach kann Pandas Zeitdifferenzen berechnen, nach Wochentagen gruppieren oder Zeitintervalle bilden.
- **Schutz bei kaputten Zeitstempeln:** Mit dem Parameter errors='coerce' werden unlesbare Zeit-Strings sicher in NaT (Not a Time) umgewandelt.

💡 **Praxis-Tipp / Merksatz:**
Erst wenn df['zeit'].dtype den Typ 'datetime64[ns]' anzeigt, sind die Zeitfunktionen von Pandas freigeschaltet!

zeit_konvertierung.py

```
import pandas as pd

df = pd.read_csv('bb12024.csv')
print('Vorher:', df['zeit'].dtype)

# Konvertierung der Text-Spalte in echte Datetime-Objekte
df['zeit'] = pd.to_datetime(df['zeit'])

print('Nachher:', df['zeit'].dtype)
print('Erster Zeitstempel:', df['zeit'].iloc[0])
print('Wochentag:', df['zeit'].dt.day_name().iloc[0])

[Ausgabe]
>>> Vorher: object
>>> Nachher: datetime64[ns]
>>> Erster Zeitstempel: 2024-01-01 00:01:00
>>> Wochentag: Monday
```


Arts Sciences
TH Köln

Die Zeitachse als Index setzen: df.set_index('zeit')

Vom nichts-sagenden Zeilenindex (0, 1, 2) zum mächtigen DatetimeIndex

Warum gehört die Zeit in den Index?

- **Standard-Index ersetzen:** Mit df.set_index('zeit', inplace=True) wird die Spalte 'zeit' zur neuen Zeilenbeschriftung (DatetimeIndex).
- **Intuitives Zeit-Slicing:** Statt Zeilennummern zu raten, schneiden wir Zeiträume direkt über Datumsangaben aus: df.loc['2024-02-10'].
- **Bereichsabfragen:** Mit df.loc['2024-02-10 08:00':'2024-02-10 12:00'] erhalten wir exakt das 4-Stunden-Fenster am Vormittag.
- **Zwingende Voraussetzung für Resampling:** Die Funktion df.resample() funktioniert nur, wenn der DataFrame einen DatetimeIndex besitzt!

💡 **Praxis-Tipp / Merksatz:**
Mit inplace=True wird der bestehende DataFrame direkt geändert, ohne dass wir df = df.set_index(...) schreiben müssen.

set_datetime_index.py

```
import pandas as pd

df = pd.read_csv('bb12024.csv')
df['zeit'] = pd.to_datetime(df['zeit'])

# Zeitspalte als Index setzen
df.set_index('zeit', inplace=True)

print(df.head(3))

# Zeitfenster-Abfrage über den neuen Index:
vormittag = df.loc['2024-01-01 08:00':'2024-01-01 08:15']
print('\nAusschnitt 08:00 bis 08:15 Uhr:')
print(vormittag[['o2_ist', 'o2_soll']])

[Ausgabe]
>>>                      o2_ist  o2_soll  nh4_mg_l  temp_c
>>> zeit
>>> 2024-01-01 00:01:00    2.14      2.0      1.85    11.4
>>> 2024-01-01 00:04:00    2.08      2.0      1.88    11.4
>>> 2024-01-01 00:07:00    2.08      2.0      1.92    11.3
```


Arts Sciences
TH Köln

Resampling: Asynchrone Sensordaten auf 5 Minuten takten

Warum unregelmäßige Zeitstempel (08:01, 08:04, 08:07) vereinheitlicht werden müssen

Wie funktioniert df.resample('5min').mean()?

- **Das Praxisproblem:** Sensoren senden oft ereignisgesteuert oder asynchron (z. B. 08:01, 08:04, 08:07). Ein direkter Vergleich mehrerer Sensoren scheitert an unterschiedlichen Zeitstempeln.
- **Feste Zeitkörbe (Bins) bilden:** df.resample('5min') teilt die Zeitachse strikt in 5-Minuten-Intervalle ein (08:00, 08:05, 08:10, 08:15, ...).
- **Aggregation mit .mean():** Alle Messwerte, die z. B. zwischen 08:00:00 und 08:04:59 liegen (also 08:01 und 08:04), werden gemittelt und dem Zeitstempel 08:00 zugeordnet.
- **Rauschunterdrückung:** Gleichzeitig glättet die Mittelwertbildung hochfrequentes Sensorrauschen.

💡 **Praxis-Tipp / Merksatz:**
Andere Intervalle sind genauso einfach: '1min' (1 Minute), '15min' (Viertelstunde), '1h' (Stunde) oder '1D' (Tag).

resampling_5min_erklaert.py

```
import pandas as pd

# Asynchrone Rohdaten (08:01, 08:04, 08:07, 08:09, 08:12)
roh = pd.DataFrame({
    'zeit': pd.to_datetime([
        '2024-01-01 08:01', '2024-01-01 08:04',
        '2024-01-01 08:07', '2024-01-01 08:09',
        '2024-01-01 08:12'
    ]),
    'o2_ist': [2.0, 2.4, 3.0, 3.2, 2.8]
}).set_index('zeit')

# Striktes Resampling auf ein 5-Minuten-Intervall
df_resampled = roh.resample('5min').mean()
print(df_resampled)

[Ausgabe]
>>>                      o2_ist
>>> zeit
>>> 2024-01-01 08:00:00     2.2  # Mittel 08:01 & 08:04
>>> 2024-01-01 08:05:00     3.1  # Mittel 08:07 & 08:09
>>> 2024-01-01 08:10:00     2.8  # Wert aus 08:12
```


Arts Sciences
TH Köln

Komplette Zeitreihen-Pipeline & Zwischenübung 3

Vom Rohdaten-Import (bb12024.csv) bis zum sauberen 5-Minuten-Datensatz

🛠️ Aufgabenstellung (Live-Übung):

- **Die 4-Schritte-Standardpipeline:** (1) CSV laden -> (2) Zeitspalte per pd.to_datetime umwandeln -> (3) Zeit als Index setzen -> (4) Auf 5min resamplen und Lücken schließen.
- **Zwischenübung 3 (15 Min.):** Schreiben Sie eine Funktion lade_und_resample(dateipfad), die bb12024.csv verarbeitet.
  - Konvertieren Sie die Spalte 'zeit' in Datetime und setzen Sie diese als Index.
  - **Führen Sie ein Resampling auf ein 5-Minuten-Intervall mit dem Mittelwert durch:** df.resample('5min').mean().
  - Schließen Sie eventuelle leere 5-Minuten-Körbe anschließend mit .interpolate() und geben Sie die ersten 4 Zeilen aus.

⏱️ **Ziel & Lösungshinweis:**
Beachten Sie die saubere Zeitachse im Output: Exakt 00:00, 00:05, 00:10, 00:15 Uhr!

loesung_stunde3_resampling.py

```
import pandas as pd

def lade_und_resample(dateipfad):
    # 1. Rohdaten einlesen
    df = pd.read_csv(dateipfad)

    # 2. Zeitstempel konvertieren & als Index setzen
    df['zeit'] = pd.to_datetime(df['zeit'])
    df.set_index('zeit', inplace=True)

    # 3. Striktes 5-Minuten-Resampling (Mittelwert)
    df_resampled = df.resample('5min').mean()

    # 4. Eventuelle Lücken linear interpolieren
    df_resampled = df_resampled.interpolate()
    return df_resampled

df_5min = lade_und_resample('bb12024.csv')
print(df_5min.head(4).round(2))

[Ausgabe]
>>>                     o2_ist  o2_soll  nh4_mg_l  temp_c
>>> zeit
>>> 2024-01-01 00:00:00    2.11      2.0      1.86    11.4
>>> 2024-01-01 00:05:00    2.08      2.0      1.92    11.3
>>> 2024-01-01 00:10:00    1.98      2.0      1.92    11.3
>>> 2024-01-01 00:15:00    2.05      2.0      1.89    11.3
```


Arts Sciences
TH Köln

STUNDE 4 (45 Min.)

Objektorientierte Programmierung (OOP) & Synthese

Reale Anlagenkomponenten als Klassen modellieren und alle Konzepte zusammenführen

Lernziele & Schwerpunkte dieser Vorlesungsstunde:

- **Paradigmenwechsel:** Warum prozedurale Skripte bei großen Systemen (viele Sensoren, Aktoren, Becken) unübersichtlich werden.
- **Grundbegriffe der OOP:** Klasse (Bauplan), Objekt/Instanz (konkretes Exemplar), Attribute (Daten) und Methoden (Funktionen).
- Der Konstruktor __init__ und die Rolle von self beim Anlegen eigener Klassen.
- **Praxisbeispiel Klasse 'Sensor':** Messwerte kapseln, mit try/except validieren und statistisch auswerten.
- **Große Synthese-Übung:** Verbindung von Funktionen, Fehlerbehandlung, Pandas-Resampling und OOP.

Prozedurale vs. Objektorientierte Programmierung (OOP)

Ein neues Paradigma: Die reale technische Welt direkt in Software-Objekten abbilden

Stunde 4 | Motivation OOP

Bisher: Prozedurale Programmierung

- **Trennung von Daten und Funktionen:** Variablen (Listen, Zahlen) und die Funktionen, die sie verarbeiten, liegen getrennt im Skript.
- **Variablen-Chaos bei vielen Komponenten:** Für 10 Sensoren entstehen lose Variablen wie sensor1_id, sensor1_typ, sensor1_wert, sensor2_id ...
- **Gefahr von Verwechslungen:** Man kann einer Funktion versehentlich die falsche Liste oder den falschen Grenzwert übergeben.
- **Gut geeignet für:** Kurze, lineare Datenanalyse-Skripte mit wenigen Komponenten.

Neu: Objektorientierte Programmierung (OOP)

- **Einheit aus Daten und Logik:** Zusammengehörige Daten (Attribute) und Funktionen (Methoden) werden in einem Objekt gebündelt.
- **Klasse (Class – Der Bauplan):** Definiert einmalig, welche Eigenschaften und Fähigkeiten ein 'Sensor' grundsätzlich hat.
- **Objekt (Instance – Das konkrete Exemplar):** Aus dem Bauplan 'Sensor' erzeugen wir beliebig viele reale Sensoren (z. B. o2_becken1, nh4_zulauf).
- **Jedes Objekt verwaltet seinen eigenen Zustand:** o2_becken1 kennt seine eigene ID, seinen Typ und seine eigene Messhistorie.

📌 **Fazit: Übrigens: Auch ein Pandas DataFrame (df) ist ein Objekt! 'df.shape' ist ein Attribut, 'df.head()' ist eine Methode.**

Technology
Arts Sciences
TH Köln

Die erste eigene Klasse und der Konstruktor (__init__)

Wie man mit 'class' einen Bauplan erstellt und Objekte mit 'self' initialisiert

Stunde 4 | Klassen & __init__

Schlüsselkonzepte beim Klassendesign

- **Schlüsselwort class:** Eingeleitet mit class Sensor: (Klassennamen schreibt man in Python immer in CamelCase mit großem Anfangsbuchstaben).
- **Der Konstruktor __init__:** Eine spezielle Methode, die automatisch aufgerufen wird, sobald ein neues Objekt dieser Klasse erzeugt wird.
- **Was bedeutet self?:** self ist die Selbstreferenz auf das gerade erzeugte konkrete Objekt – damit weiß Python, wessen Attribut gesetzt wird.
- **Attribute anlegen:** Mit self.id = sensor_id und self.wert = 0.0 speichern wir die Eigenschaften dauerhaft im Objekt.

💡 **Praxis-Tipp / Merksatz:**
Merke: 'self' muss bei jeder Methode innerhalb einer Klasse als erster Parameter in der Klammer stehen, wird beim Aufruf aber automatisch übergeben!

klasse_sensor_konstruktor.py

```
class Sensor:
    """Bauplan für einen industriellen Messsensor."""

    def __init__(self, sensor_id, typ, einheit):
        # Attribute des Objekts initialisieren
        self.id = sensor_id
        self.typ = typ
        self.einheit = einheit
        self.aktueller_wert = 0.0
        self.historie = []

# Zwei konkrete Objekte nach dem Bauplan erzeugen
s1 = Sensor(sensor_id=101, typ='O2', einheit='mg/L')
s2 = Sensor(sensor_id=102, typ='Temperatur', einheit='°C')

print(f'Sensor {s1.id}: Typ={s1.typ} ({s1.einheit})')
print(f'Sensor {s2.id}: Typ={s2.typ} ({s2.einheit})')

[Ausgabe]
>>> Sensor 101: Typ=O2 (mg/L)
>>> Sensor 102: Typ=Temperatur (°C)
```


Arts Sciences
TH Köln

Methoden definieren: Objekten Leben einhauchen

Kombination von OOP mit der Fehlerbehandlung (try/except) aus Stunde 1

Methoden sind Funktionen innerhalb einer Klasse

- **Zustandsänderung über Methoden:** Statt Attribute von außen unkontrolliert zu überschreiben, nutzen wir eine Methode aktualisiere_wert(self, roh_wert).
- **Integrierte Schutzlogik:** Die Methode prüft per try / except ValueError selbstständig, ob der neue Messwert gültig ist!
- **Messhistorie führen:** Jeder gültige Wert wird automatisch an die objekt-eigene Liste self.historie angehängt.
- **Auswertungsmethode:** Die Methode mittelwert(self) berechnet jederzeit den Durchschnitt aller bisherigen Messungen dieses Sensors.

💡 **Praxis-Tipp / Merksatz:**
Hier greift alles ineinander: Das Sensor-Objekt kapselt seine Daten UND schützt sich selbst vor fehlerhaften Eingaben!

sensor_mit_methoden.py

```
class Sensor:
    def __init__(self, sensor_id, typ):
        self.id = sensor_id
        self.typ = typ
        self.wert = 0.0
        self.historie = []

    def aktualisiere_wert(self, roh_input):
        try:
            self.wert = float(roh_input)
            self.historie.append(self.wert)
        except ValueError:
            print(f'[Sensor {self.id}] Ignoriert: {roh_input}')

    def mittelwert(self):
        if not self.historie:
            return 0.0
        return round(sum(self.historie) / len(self.historie), 2)

o2_sensor = Sensor(sensor_id=101, typ='O2')
for m in ['4.1', '4.5', 'Error', '4.3']:
    o2_sensor.aktualisiere_wert(m)
print('Aktuell:', o2_sensor.wert, '| MW:', o2_sensor.mittelwert())

[Ausgabe]
>>> [Sensor 101] Ignoriert: Error
>>> Aktuell: 4.3 | MW: 4.3
```


Technology, Arts Sciences TH Köln

Große Abschlussübung: Pandas & OOP kombinieren

Ein objektorientierter Becken-Analysator für den Datensatz 'bb12024.csv'

Stunde 4 | Synthese-Übung

🛠️ Aufgabenstellung (Live-Übung):

- **Ziel der Abschlussübung:** Verbinden Sie die Konzepte aus allen 4 Vorlesungsstunden in einer einzigen sauberen Klasse BeckenAnalyse.
- **1. Konstruktor (__init__):** Speichert den Dateipfad (bb12024.csv) und legt das Attribut self.df = None an.
- **2. Methode aufbereiten_5min(self):** Liest die CSV mit pd.read_csv() ein, wandelt 'zeit' mit pd.to_datetime() um, setzt den Index und führt das 5-Minuten-Resampling inkl. Interpolation durch.
- **3. Methode zeige_report(self):** Gibt den Mittelwert von 'o2_ist' und 'temp_c' formatiert auf der Konsole aus.

⏱️ **Ziel & Lösungshinweis:**
Mit nur 3 Zeilen im Hauptprogramm lassen sich nun beliebig viele Becken-Datensätze standardisiert auswerten!

loesung_stunde4_synthese.py

```
import pandas as pd

class BeckenAnalyse:
    def __init__(self, becken_name, csv_pfad):
        self.name = becken_name
        self.pfad = csv_pfad
        self.df = None

    def aufbereiten_5min(self):
        roh_df = pd.read_csv(self.pfad)
        roh_df['zeit'] = pd.to_datetime(roh_df['zeit'])
        roh_df.set_index('zeit', inplace=True)
        self.df = roh_df.resample('5min').mean().interpolate()

    def zeige_report(self):
        o2_mw = self.df['o2_ist'].mean()
        t_mw = self.df['temp_c'].mean()
        print(f'=== Report {self.name} (5min-Takt) ===')
        print(f'Zeilen: {len(self.df)} | O2: {o2_mw:.2f} | T:
{t_mw:.1f}°C')

bb1 = BeckenAnalyse('Belebungsbecken 1', 'bb12024.csv')
bb1.aufbereiten_5min()
bb1.zeige_report()

[Ausgabe]
>>> === Report Belebungsbecken 1 (5min-Takt) ===
>>> Zeilen: 105120 | O2: 1.84 | T: 15.6°C
```


Arts Sciences
TH Köln

Zusammenfassung & Werkzeugkasten für das Projekt

Was wir in diesen 4 Vorlesungsstunden für reale Digitalisierungsprojekte gelernt haben

Zusammenfassung

Rückblick: Stunde 1 & Stunde 2

- **DRY-Prinzip & Funktionen (Stunde 1):** Wiederkehrende Berechnungen (wie berechne_mittelwert) kapseln wir mit def und return statt Copy-Paste-Schleifen.
- **Scope verstehen (Stunde 1):** Lokale Variablen (wie summe) leben nur innerhalb der Funktion und schützen das Hauptprogramm vor Namenskonflikten.
- **Robuste Fehlerbehandlung (Stunde 1):** Mit try und except ValueError fangen wir defekte Sensor-Strings ('Error') ab, ohne dass das Programm abstürzt.
- **Pandas DataFrames (Stunde 2):** Mit pd.read_csv('bb12024.csv'), df.head(), df.info() und df.describe() untersuchen wir große, heterogene Messdatensätze in Sekunden.

Rückblick: Stunde 3 & Stunde 4

- **Datenbereinigung (Stunde 3):** Datenlücken (NaN) diagnostizieren wir mit df.isna().sum() und beheben sie gezielt über dropna(), ffill() oder interpolate().
- **Zeitreihen & 5-Minuten-Resampling (Stunde 3):** Mit pd.to_datetime(), set_index('zeit') und df.resample('5min').mean() bringen wir asynchrone Sensoren auf einen exakten Zeittakt.
- **Objektorientierung (Stunde 4):** Klassen (class Sensor) bündeln Daten (Attribute) und Logik (Methoden) über __init__ und self zu wartbaren Software-Bausteinen.
- **Nächster Schritt:** Direkte Anwendung dieser Pipeline auf Ihre eigenen Messdaten im Semesterprojekt!

📌 **Fazit: Clean Code + Pandas-Zeitreihen + OOP = Das Fundament für professionelle Datenanalyse und Automatisierung in Python.**

Technology
Arts Sciences
TH Köln

Fragen?

